perhaps ways it could be extended further or made more convenient. Here are a few ideas to get you started:

- Write properties to specify the behavior of some of the other functions we wrote for `List` and `Stream`, for instance, `take`, `drop`, `filter`, and `unfold`.
- Write a sized generator for producing the `Tree` data type defined in chapter 3, and then use this to specify the behavior of the `fold` function we defined for `Tree`. Can you think of ways to improve the API to make this easier?
- Write properties to specify the behavior of the `sequence` function we defined for `Option` and `Either`.

8.6 The laws of generators

Isn't it interesting that many of the functions we've implemented for our `Gen` type look quite similar to other functions we defined on `Par`, `List`, `Stream`, and `Option`? As an example, for `Par` we defined this:

```
def map[A,B](a: Par[A])(f: A => B): Par[B]
```

And in this chapter we defined `map` for `Gen` (as a method on `Gen[A]`):

```
def map[B](f: A => B): Gen[B]
```

We've also defined similar-looking functions for `Option`, `List`, `Stream`, and `State`. We have to wonder, is it merely that our functions share similar-looking signatures, or do they satisfy the same *laws* as well? Let's look at a law we introduced for `Par` in chapter 7:

```
map(x)(id) == x
```

Does this law hold for our implementation of `Gen.map`? What about for `Stream`, `List`, `Option`, and `State`? Yes, it does! Try it and see. This indicates that not only do these functions share similar-looking signatures, they also in some sense have analogous meanings in their respective domains. It appears there are deeper forces at work! We're uncovering some fundamental patterns that cut across domains. In part 3, we'll learn the names for these patterns, discover the laws that govern them, and understand what it all means.

8.7 Summary

In this chapter, we worked through another extended exercise in functional library design, using the domain of property-based testing as inspiration.

We reiterate that our goal was not necessarily to learn about property-based testing as such, but to highlight particular aspects of functional design. First, we saw that oscillating between the abstract algebra and the concrete representation lets the two inform each other. This avoids overfitting the library to a particular representation, and also avoids ending up with a floating abstraction disconnected from the end goal.

Second, we noticed that this domain led us to discover many of the same combinators we've now seen a few times before: `map`, `flatMap`, and so on. Not only are the signatures of these functions analogous, the *laws* satisfied by the implementations are analogous too. There are a great many seemingly distinct *problems* being solved in the world of software, yet the space of functional *solutions* is much smaller. Many libraries are just simple combinations of certain fundamental structures that appear over and over again across a variety of different domains. This is an opportunity for code reuse that we'll exploit in part 3, when we learn both the names of some of these structures and how to spot more general abstractions.

In the next and final chapter of part 2, we'll look at another domain, *parsing*, with its own unique challenges. We'll take a slightly different approach in that chapter, but once again familiar patterns will emerge.